output. As the *Map()* phase happens across a very large distributed dataset, spread across a huge cluster of nodes, it is subsequently run through a *Reduce()* phase which aggregates the sorted dataset, coming in from multiple map nodes. This framework, along with the underlying HDFS system, enables processing of very large datasets running into Petabytes, spread across thousands of nodes.

Apache Flink *(https://flink.apache.org/)* is a data processing system that combines the scalability and power of the Hadoop HDFS layer along with the declarations and optimisations that are the cornerstone of relational database systems. Flink provides a runtime system, which is an alternative to the Hadoop MapReduce framework.

Apache Tez *(https://tez.apache.org/)* is a distributed data processing engine that sits on top of Yarn (Hadoop 2.0 ecosystem). Tez models the data processing workflows as Distributed Acyclic Graphs (DAGs). With this distinctive feature, Tez allows developers to intuitively model their complex data processing jobs as a pipeline of tasks, while leveraging the underlying resource management capabilities of the Hadoop 2.0 ecosystem.

Apache Spark *(https://spark.apache.org/)* is a distributed execution engine for Big Data processing that provides efficient abstractions to process large datasets in memory. While MapReduce on Yarn provides abstraction for using a cluster's computational resources, it lacks efficiency for iterative algorithms and interactive data mining—algorithms that need to reuse data in between computations. Spark implements in-memory fault tolerant data abstractions in the form of RDDs (Resilient Distributed Datasets), which are parallel data structures stored in memory. RDDs provide fault-tolerance by tracking transformations (lineage) rather than changing actual data. In case a partition has to be recovered after loss, the transformations need to be applied on just that dataset. This is far more efficient than replicating datasets across nodes for fault tolerance, and this is supposedly 100x faster than Hadoop MR.

Spark also provides a unified framework for batch processing, stream data processing, interactive data mining and includes APIs in Java, Scala and Python. It provides an interactive shell for faster querying capabilities, libraries for machine learning (MLLib and GraphX), an API for graph data processing, SparkSQL (a declarative query language), and SparkStreaming (a streaming API for stream data processing).

SparkStreaming is a system for processing event streams in real-time. SparkStreaming treats streaming as processing of datasets in micro-batches. The incoming stream is divided into batches of configured number of seconds. These batches are fed into the underlying Spark system and are processed the same way as in the Spark batch programming paradigm. This makes it possible to achieve the very low latencies needed for stream processing, and at the same time integrate batch processing with real-time stream processing.

Apache Storm *(https:/storm.apache.org/)* is a system for processing continuous streams of data in real-time. It is highly scalable, fault tolerant, and ensures the notion of guaranteed processing so that no events are lost. While Hadoop provides the framework for batch processing of data, Storm does the same for streaming event data.

It provides Directed Acyclic Graph (DAG) processing for defining the data processing pipeline or topology using a notion of spouts (input data sources) and bolts. Streams are tuples that flow through these processing pipelines.

A Storm cluster consists of three components:

- **Nimbus,** which runs on the master node and is responsible for distribution of work amongst the worker processes.
- **Supervisor** daemons run on the worker nodes, listen to the tasks assigned, and manage the worker processes to start/stop them as needed, to get the work done.
- **Zookeeper** handles the co-ordination between Nimbus and Supervisors, and maintains the state for fault-tolerance.

Higher level languages for analytics and querying

As cluster programming frameworks evolved to solve the Big Data processing problems, another problem started to emerge as more and more real life use cases were attempted. Programming using these computing frameworks got increasingly complex, and became difficult to maintain. The skill scalability became another matter of concern, as there were a lot of people available with domain expertise familiar with skills such as SQL and scripting. As a result, higher level programming abstractions for the cluster computing frameworks began to emerge, that abstracted the low level programming APIs. Some of these frameworks are discussed in this section.

Hive *(https:/hive.apache.org/)* and **Pig** *(https:/pig.apache.org/)* are higher level language implementations for MapReduce. The language interface internally generates MapReduce programs from the queries written in the high level languages, thereby abstracting the underlying nitty-gritty of MapReduce and HDFS.

While Pig implements PigLatin, which is a procedural-like language interface, Hive provides the Hive Query Language (HQL), which is a declarative and SQL-like language interface.

Pig lends itself well to writing data processing pipelines for iterative processing scenarios. Hive, with a declarative SQL-like language, is more usable for ad hoc data querying, explorative analytics and BI.

BlinkDB *(http:/blinkdb.org/)* is a recent entrant into the Big Data processing ecosystem. It provides a platform for interactive query processing that supports approximate